

Decision Tree Classifier / Regressor

criterion = "gini" (default), "entropy", "log-loss"
(for classifier)

criterion = "squared-error" (default), "absolute-error",
(for regressor) "friedman-mse", "poisson".

⇒ we generally use "gini" when it comes to classifier in modern days.

↳ For regressor, "Squared-error" is impurity = $E(y - \bar{y})^2$ is aimed to be reduced. It is sensitive to outliers.

↳ "absolute-error" is ~~the~~ Mean Absolute Error

$MAE = \frac{1}{n} \sum |y - \hat{y}|$. It is not sensitive to outliers.
The prediction ~~also~~ for a leaf becomes the median instead of mean.

↳ "friedman-mse" is used in Gradient Boosting Trees (GBRT). It computes improvement in loss using the Friedman criterion. $\text{Gain} = L^2/p$.
less common for standalone Decision Tree Regressor
slightly faster than squared-error during boosting.

↳ "poisson" is used when target variable represents counts by exponential growth patterns

The model requires, $y \geq 0$

$$\text{Loss} = y \cdot \log\left(\frac{y}{\hat{y}}\right) - (y - \hat{y})$$

Not for negative or continuous floating regression problems.

splitter : "best" (default) , "random"
⇒ "best" : tries all possible midpoints between distinct values as thresholds and picks the threshold that gives the best split.

↳ "random" picks some random values from features and uses those as potential splitting thresholds and evaluates impurity reduction only for those.

↳ Random is used for reducing overfitting by introducing randomness

max_depth : it is maximum tree depth (the max_depth is length with longest path)

It is most important parameter for reducing overfitting (pre-pruning)

min_samples_split : Maximum samples required to split a node (Default = 2)

→ Higher → less splits → simple tree.

min_samples_leaf : Minimum samples at a leaf node

→ Higher → more generalization (reduce overfitting)

min_weight_fraction_leaf = use when data has class imbalance, misclassification cost, etc.

example:

Passenger	Age	Sex	Survived
P1	7	F	1
P2	65	M	0
P3	30	F	1
P4	22	M	0
P5	2	F	1
P6	25	M	0

weight rule:

Age $\leq 18 \rightarrow$ weight = 2 (more important)

Age $> 18 \rightarrow$ weight = 1 (normal)

Total weight = $2 + 1 + 1 + 1 + 2 + 1 = 8$

min. weight fraction = leaf = 0.25

25% of 8 is 2 (minimum)

example:-

left node (age ≤ 10):

Passenger

Age

weight

P1

7

2

P5

2

2

left weight = $2 + 2 = 4$ ($4 > 2$)

Right node (age > 10):

Passenger

Age

weight

P2

65

1

P3

30

1

P4

22

1

P6

25

1

Right leaf weight = $1 + 1 + 1 + 1 = 4$ ($4 > 2$)

when the leaf weight will be less than 2,
no further splitting would occur.

max. features: "auto", "sqrt", "log 2"

↳ It reduces correlation between splits, speeds training.

max. leaf nodes: smaller = simpler model

min. impurity-decrease: Node will split only if
impurity decrease $>$ threshold.

class-weight (classifier only): It helps with
class imbalance

Normally, impurity is computed like this:-

$$Gini = 1 - \sum (p_i^2)$$

class-weight is used, the probabilities are weighted.

$$Gini\text{-weighted} = 1 - \sum (w_i \cdot p_i^2)$$

w_i = weight of class i

p_i = proportion of samples of class i in that node.

$$w_i = \frac{n\text{-samples}}{n\text{-classes} \cdot n\text{-samples-in-class-}i}$$

class-weight = "balanced" or $\{0:1, 1:5\}$

(0,1) are columns
(1,5) are weights

ccp-alpha

cost-complexity pruning parameter
(post-pruning)

Removes weak subtrees → reduces overfitting.

random-state

Ensures reproducibility